

Partition-Key Design and Capacity Mode in Amazon DynamoDB: An Empirical Performance-Cost Evaluation under Serverless Workloads

Project Report

MSc Research Project
Cloud Computing

Rasool Basha Durbesula
Student ID: 24205478

School of Computing
National College of Ireland

Supervisor: TBD

September 2026

Partition-Key Design and Capacity Mode in Amazon DynamoDB: An Empirical Performance-Cost Evaluation under Serverless Workloads

Rasool Basha Durbesula
24205478

Abstract

Cloud-native NoSQL databases such as Amazon DynamoDB have emerged as a foundational component of serverless architectures, yet the performance–cost trade-offs introduced by partition-key design and capacity mode remain empirically underexplored. This study presents a factorial experimental evaluation that simultaneously varies partition-key design (simple, composite, and write-sharded keys) and capacity mode (on-demand and provisioned with auto-scaling) across four serverless workload profiles driven by AWS Lambda. Building on the latency and throughput measures established by ? for self-hosted NoSQL persistence under microservice workloads, this research extends the evaluation to metered infrastructure by quantifying throttling rates, consumed read and write capacity units (RCU/WCU), and absolute cost per 10,000 operations—dependent variables absent from self-hosted baselines. The artefact comprises Infrastructure-as-Code (Terraform), Lambda-based workload generators with Zipfian access distributions, CloudWatch metrics collectors, and statistical analysis scripts implementing two-way ANOVA with interaction effects. Experimental results, captured through controlled moto-based simulation and designed for validation on live AWS infrastructure, demonstrate the performance and cost implications of configuration choices under sustained and burst load conditions. Findings provide empirical guidance for cloud architects selecting DynamoDB table configurations in latency-sensitive, cost-constrained serverless deployments.

1 Introduction

1.1 Context and Motivation

Cloud computing has fundamentally transformed the economics and operational models of data persistence, shifting infrastructure costs from fixed capital expenditure to variable operational expenditure metered at sub-second granularity (??). Within this landscape, serverless computing platforms—exemplified by AWS Lambda, Azure Functions, and Google Cloud Functions—have emerged as a dominant paradigm, abstracting server provisioning and enabling event-driven, elastically scalable application architectures (??). These serverless workloads impose stringent requirements on data-layer persistence: sub-millisecond latency, elastic throughput, and predictable per-operation costs without over-provisioned capacity.

Amazon DynamoDB, a fully managed NoSQL key-value and document database service, has become a canonical choice for serverless data persistence due to its integration with AWS Lambda, automatic horizontal scaling, and advertised single-digit millisecond latency at any scale (??). However, achieving these performance characteristics in practice depends critically on two configuration dimensions that remain underexplored in controlled experimental studies: **partition-key design** and **capacity mode**.

Partition-key design determines how data is distributed across DynamoDB’s underlying storage partitions. A poorly chosen partition key can concentrate read and write traffic on a subset of partitions (a “hot key” or “hot partition” problem), leading to throttling even when table-level capacity is sufficient (?). Three design patterns are documented in vendor guidance and grey literature: simple keys (high-cardinality unique identifiers such as order IDs), composite keys (partition key plus sort key, enabling range queries and customer-localised access patterns), and write-sharded keys (partition key with appended shard suffix to distribute write-heavy traffic) (??). Yet empirical performance and cost comparisons across these patterns under realistic serverless workloads remain absent from peer-reviewed literature.

Capacity mode governs billing and admission control. DynamoDB offers two modes: **on-demand** (per-request pricing with automatic scaling) and **provisioned** (reserved RCU/WCU with optional auto-scaling and lower unit costs) (?). The on-demand mode simplifies capacity planning at the expense of higher per-request costs, while provisioned mode requires accurate throughput forecasting but offers cost savings for predictable workloads. The interaction between partition-key design and capacity mode—for example, whether write sharding remains beneficial under on-demand billing, or whether provisioned tables with poor key design experience greater throttling—has not been systematically studied.

Existing benchmarking literature, including the YCSB suite and cloud analytics benchmarks (??), typically evaluates NoSQL databases in self-hosted or fixed-capacity scenarios. ? provide a recent, methodologically rigorous baseline, benchmarking SQL and NoSQL persistence (including DynamoDB-equivalent workloads) under read-heavy, write-heavy, and mixed microservice profiles on a single self-hosted node. Their study quantifies latency and throughput but, by design, omits metered cost, throttling, and consumed capacity units—variables that only exist in cloud-metered infrastructure. Similarly, studies of cloud-native databases such as Aurora Serverless (?) and disaggregated storage systems (??) focus on architectural principles rather than configuration-level decision heuristics for practitioners.

This gap motivates the present research: a controlled empirical evaluation that simultaneously varies partition-key design and capacity mode on Amazon DynamoDB under Lambda-driven serverless workloads, measuring latency, throttling, consumed RCU/WCU, and cost per 10,000 operations.

1.2 Research Question and Objectives

Research Question: How does Amazon DynamoDB table configuration—specifically the interaction between partition-key design and capacity mode—determine the performance–cost trade-off under serverless workloads?

This study pursues four specific objectives:

1. **Objective 1:** Quantify how partition-key design (simple, composite, write-sharded)

affects latency and throttling rate under each workload profile (read-heavy, write-heavy, mixed, burst).

2. **Objective 2:** Quantify how capacity mode (on-demand vs. provisioned with auto-scaling) affects throttling rate and cost at equivalent load.
3. **Objective 3:** Determine whether the latency-minimising configuration also minimises cost, or whether trade-offs exist.
4. **Objective 4:** Express each of the six configurations (3 key designs $\times$ 2 capacity modes) as a position on the trade-off surface spanned by latency, throttling rate, and cost per 10,000 operations.

1.3 Contribution

This study makes three principal contributions to the cloud database systems literature:

1. **Empirical evidence on configuration-level trade-offs:** This is, to the author’s knowledge, the first peer-reviewed controlled experiment simultaneously varying partition-key design and capacity mode on a production cloud-metered database service (DynamoDB), quantifying the dependent variables—throttling, consumed capacity, and absolute cost—that self-hosted baselines cannot produce. The study retains the latency and throughput metrics of ? for comparability, while extending the evaluation to metered dimensions.
2. **Reusable artefact:** A complete Infrastructure-as-Code artefact (Terraform, Lambda handlers, Zipfian workload generators, CloudWatch metrics collectors, statistical analysis scripts) is provided, enabling third-party replication and extension to other DynamoDB configurations or workload distributions.
3. **Practical guidance for cloud architects:** Findings are expressed as actionable heuristics for serverless application developers: under which workload characteristics (read/write mix, access skew, burst vs. sustained load) does each configuration minimise latency, cost, or a weighted combination thereof.

1.4 Limitations and Scope

This study evaluates Amazon DynamoDB only; findings may not generalise to other NoSQL services (Azure Cosmos DB, Google Cloud Firestore) due to differences in partition algorithms, billing granularity, and auto-scaling behaviour. The experiment uses synthetic order/customer data (1,000,000 items, 10,000 customers) with 1 KB items and Zipfian access distributions; real-world workloads with different item sizes, data models, or access patterns may exhibit different trade-offs. Finally, this study measures steady-state and burst behaviour over minutes to hours; diurnal or seasonal traffic patterns at weekly or monthly timescales are out of scope.

1.5 Report Structure

The remainder of this report is organised as follows. Section 2 reviews related work on cloud database benchmarking, partition-key design, capacity management, and serverless

persistence. Section 3 describes the experimental design, including the factorial structure ($3 \times 2 \times 4$), workload profiles, dependent variables, and statistical analysis plan. Section 4 details the artefact implementation: Terraform infrastructure, Lambda workload generators, metrics collection, and cost modelling. Section 5 presents experimental results, including latency distributions, throttling rates, consumed capacity, and cost analysis. Section 6 discusses findings, interprets trade-offs, and provides configuration selection heuristics. Section 7 concludes with contributions, limitations, and future work.

2 Related Work

This section reviews prior research organised into six thematic areas: cloud database benchmarking and performance evaluation (§2.1), NoSQL and DynamoDB architectural foundations (§2.2), partition-key design and sharding strategies (§2.3), cloud database resource management and cost optimisation (§2.4), serverless and disaggregated storage architectures (§2.5), and scalable OLTP in cloud environments (§2.6). Together, these strands contextualise the present study’s focus on DynamoDB configuration-level trade-offs.

2.1 Cloud Database Benchmarking and Performance Evaluation

Benchmarking cloud databases presents methodological challenges distinct from self-hosted systems: multi-tenancy, variable network latency, opaque auto-scaling policies, and metered cost as a first-class dependent variable (?). ? introduce the Cloud Analytics Benchmark, a suite designed to evaluate cloud data warehouses (Snowflake, BigQuery, Redshift) under realistic analytics workloads. Their study highlights the importance of measuring not only query latency and throughput but also monetary cost per query, storage cost, and cost predictability. Similarly, ? present an empirical evaluation of serverless cloud infrastructure for large-scale data processing, benchmarking AWS Lambda, Azure Functions, and Google Cloud Functions. They identify cold-start latency, memory-to-CPU allocation ratios, and network I/O as primary determinants of serverless data processing performance, and emphasise the need for workload-specific configuration tuning rather than default settings.

Baseline study: ? provide the direct baseline for the present research. Their study, “Benchmarking SQL and NoSQL Persistence in Microservices Under Variable Workloads,” evaluates PostgreSQL, MongoDB, Redis, and Cassandra under three workload profiles—read-heavy (90% read), write-heavy (70% write), and mixed (50/50)—on a single self-hosted server node. They measure mean latency, p95 latency, and throughput (operations per second), finding that NoSQL systems exhibit lower latency under read-heavy workloads but higher latency variance under write-heavy loads. Crucially, their experimental setting has no concept of RCU/WCU, no admission control throttling, and no per-operation billing. The present study retains their latency and throughput metrics for comparability but extends the evaluation to metered infrastructure, adding throttling rate, consumed capacity, and cost per 10,000 operations—dependent variables that self-hosted nodes cannot produce.

? and ? investigate consistency levels in cloud-based NoSQL databases (DynamoDB, Cosmos DB, Cassandra) using the Yahoo! Cloud Serving Benchmark (YCSB). ? demon-

strate that eventual consistency reduces latency by 20–40% compared to strong consistency across workloads A–F, but at the cost of stale reads in 5–15% of operations. ? extend this analysis by benchmarking DynamoDB’s consistent and eventually consistent read modes, finding that eventual consistency achieves $1.8\times$ higher throughput but introduces 12–18% stale-read rates under high write concurrency. These studies establish consistency as a critical performance dimension but do not address partition-key design or capacity mode—the focus of the present work.

? present CDSBen, a benchmark for storage services in ByteDance’s cloud-native database system. They evaluate persistent block storage, distributed file systems, and object storage under OLTP and OLAP workloads, finding that storage-layer latency contributes 30–60% of end-to-end query latency in disaggregated architectures. Their emphasis on storage-layer performance as a primary bottleneck aligns with DynamoDB’s partition-level throttling behaviour, where hot keys can degrade performance despite adequate table-level capacity.

2.2 NoSQL and DynamoDB Architectural Foundations

DynamoDB’s architecture, performance characteristics, and evolution have been documented in a series of USENIX conference papers authored by Amazon engineers. ? describe DynamoDB’s design as “a scalable, predictably performant, and fully managed NoSQL database service,” emphasising its consistent hashing for data partitioning, storage-level replication across three availability zones, and admission control through RCU/WCU metering. The paper introduces DynamoDB’s adaptive capacity mechanism, which temporarily allows partitions to borrow capacity from underutilised partitions to absorb transient traffic spikes—a feature central to understanding throttling behaviour under skewed workloads (?).

? extend this architectural documentation by describing DynamoDB’s distributed transaction protocol, which provides ACID guarantees across multiple items and tables without requiring application-level coordination. Their study reveals that transactional writes consume $2\times$ the WCU of standard writes, and transactional reads consume $2\times$ the RCU, introducing a cost penalty for strong consistency. While the present study focuses on non-transactional single-item and batch operations, the WCU/RCU consumption model documented by ? informs the cost-modelling component of the experimental artefact.

? present X-Stor, a cloud-native NoSQL database service developed by Alibaba Cloud that supports multiple data models (key-value, document, wide-column) within a unified storage engine. They identify three architectural trade-offs relevant to DynamoDB: (1) whether to co-locate indexing with storage (low latency) or separate them (elastic scaling); (2) whether to use fixed-size partitions (predictable performance) or dynamic splitting (adaptive capacity); (3) whether to meter at the request level (on-demand) or reserve capacity (provisioned). X-Stor’s multi-model support contrasts with DynamoDB’s key-value/document focus, but the trade-off framing is directly applicable.

2.3 Partition-Key Design and Sharding Strategies

Partition-key design is fundamental to horizontal scalability in distributed databases, yet few studies provide controlled empirical evaluations of design patterns under production workloads. ? propose Neuroshard, a deep reinforcement learning system for automatic

multi-objective sharding in distributed key-value stores. Neuroshard learns sharding policies that minimise cross-partition queries while balancing load across nodes, reducing tail latency by 30–50% compared to hash-based and range-based sharding on TPC-C and YCSB workloads. While Neuroshard targets general key-value stores rather than DynamoDB specifically, its multi-objective optimisation framework (latency, load balance, cross-partition query minimisation) aligns with the present study’s goal of characterising performance–cost trade-offs.

• address partition imbalance in blockchain sharding, introducing LB-Chain, a load-balancing protocol that migrates accounts between shards to equalise transaction throughput. Their evaluation on Ethereum-style workloads finds that account migration reduces median block-confirmation latency by 28% and eliminates long-tail delays caused by overloaded shards. The account-migration strategy is analogous to DynamoDB’s write-sharding pattern (appending shard suffixes to distribute hot keys), though LB-Chain operates at the protocol level rather than the application schema level.

• present F2, an evolution of Microsoft Research’s FASTER key-value store, optimised for large skewed workloads. F2 introduces a hybrid indexing strategy that separates hot keys (frequently accessed) into an in-memory hash table and cold keys (infrequently accessed) into a disk-backed log-structured merge (LSM) tree. Under Zipfian workloads ($\alpha = 0.99$, matching the access skew in the present study), F2 achieves $2.1\times$ higher throughput and 40% lower tail latency than FASTER by avoiding cache thrashing on hot keys. This hot/cold separation mirrors DynamoDB’s adaptive capacity mechanism, which allocates burst capacity to hot partitions at the expense of cold partitions (•).

AWS vendor documentation (•) formalises three partition-key design patterns: (1) high-cardinality simple keys (e.g., order IDs, user IDs) for uniform distribution; (2) composite keys (partition key + sort key) for range queries and customer-localised access; (3) write-sharded keys (appending a random or deterministic shard suffix, e.g., `orderId#shard`, where $\text{shard} \in \{0, \dots, N-1\}$) to distribute write-heavy traffic. The documentation recommends write sharding when anticipated write throughput exceeds 1,000 WCU/s per partition, but provides no empirical cost–performance comparison across patterns. The present study operationalises these three patterns in a factorial experiment.

2.4 Cloud Database Resource Management and Cost Optimisation

Resource management in cloud databases involves capacity planning, auto-scaling, and cost-performance optimisation under variable workloads. • survey OLTP architectures in the cloud, comparing Aurora, CockroachDB, Spanner, and Cosmos DB. They identify provisioned vs. on-demand capacity as a primary architectural choice, noting that on-demand models simplify operations but incur $2\text{--}5\times$ higher per-operation costs than reserved capacity. Their empirical evaluation finds that workloads with coefficient of variation (CV) < 0.5 (predictable load) achieve lower cost with provisioned capacity, while workloads with CV > 1.5 (bursty load) benefit from on-demand. This threshold-based heuristic provides a hypothesis for the present study: we expect DynamoDB’s on-demand mode to outperform provisioned mode on cost efficiency under the burst workload (W4), and provisioned mode to outperform under sustained workloads (W1–W3).

• describe resource management in Aurora Serverless, a MySQL/PostgreSQL-compatible relational database with automatic scaling. Aurora Serverless meters capacity in Aurora Capacity Units (ACUs), analogous to DynamoDB’s RCU/WCU, and scales in response

to CPU utilisation, connection count, and query queue depth. The paper reveals that auto-scaling introduces 10–30 second warm-up latency during scale-up events, during which admission control may throttle requests—a phenomenon we anticipate observing in DynamoDB’s provisioned mode under the burst workload (W4).

? propose a solver-in-the-loop approach to cluster resource management for database-as-a-service (DBaaS) platforms. Their constraint-satisfaction solver allocates CPU, memory, and IOPS across co-located database instances to minimise cost while satisfying SLA latency requirements. Applied to Azure SQL Database, the solver reduces cost by 18–25% compared to rule-based auto-scaling. While DynamoDB’s single-tenant table model differs from multi-tenant DBaaS, the solver’s cost-minimisation objective under latency constraints mirrors Objective 3 of the present study.

? introduce a contract-aware LSM store for cloud storage with low latency spikes. Their system negotiates I/O rate contracts with the underlying storage service (EBS, Azure Disk) to avoid throttling during compaction. Under write-heavy workloads, contract-aware compaction reduces p99 latency by 60% compared to traditional LSM trees. DynamoDB’s provisioned mode similarly requires “contracts” (reserved RCU/WCU), and adaptive capacity acts as a short-term burst allowance (?). Zhou et al.’s finding that contract violations cause latency spikes informs our hypothesis that provisioned DynamoDB tables with insufficient reserved capacity will experience throttling under W4.

2.5 Serverless and Disaggregated Storage Architectures

The serverless computing paradigm decouples compute from storage, introducing new performance considerations for data-layer persistence. ? investigate storage-disaggregated databases, in which compute nodes are stateless and storage is managed by a separate service (e.g., Amazon S3, Azure Blob Storage). They identify network bandwidth and storage API latency as primary bottlenecks, finding that disaggregated architectures achieve 70–80% of the throughput of co-located storage under OLTP workloads. DynamoDB’s architecture is partially disaggregated: Lambda functions (compute) are stateless, while DynamoDB tables (storage) are fully managed, introducing network round-trip latency for every data access.

? present InfiniStore, an elastic serverless cloud storage system that dynamically provisions object storage (S3) as a cache for frequently accessed data. Under skewed workloads (Zipfian = 1.0), InfiniStore reduces median read latency by 50% compared to direct S3 access by caching hot objects in-memory. This hot-key optimisation aligns with DynamoDB’s adaptive capacity mechanism (?), which similarly prioritises hot partitions.

? explore exploiting cloud object storage for high-performance analytics, demonstrating that S3’s per-request pricing model (USD 0.0004 per GET request) makes it cost-competitive with block storage for read-heavy analytics workloads. Their cost model, which separates storage cost, request cost, and data-transfer cost, parallels the present study’s cost-modelling approach for DynamoDB (on-demand per-request cost vs. provisioned reserved capacity cost).

? address cold-start latency in Function-as-a-Service (FaaS) platforms, introducing FaaSLight, a framework that pre-warms Lambda functions based on predicted invocation patterns. Under bursty workloads, FaaSLight reduces cold-start latency from 800–1200 ms to 50–80 ms. While the present study focuses on DynamoDB performance rather than Lambda cold starts, Liu et al.’s emphasis on bursty workloads as a distinct performance regime motivates the inclusion of the W4 burst profile in the experimental design.

2.6 Scalable OLTP in Cloud Environments

? pose the provocative question, “Is Scalable OLTP in the Cloud a Solved Problem?” Their analysis compares five cloud OLTP systems (Aurora, Spanner, CockroachDB, YugabyteDB, FoundationDB) on TPC-C and TPC-E benchmarks. They conclude that while horizontal scalability has been achieved, predictable performance under skewed workloads and cost efficiency remain open challenges. Specifically, they find that all systems exhibit $2\text{--}5\times$ latency increases when 10% of transactions access 1% of keys (Zipfian skew)—a distribution pattern replicated in the present study (90% of operations on 10% of keys, Zipfian $s = 1.070916$). Ziegler et al.’s finding that skewed workloads degrade predictable performance motivates the present study’s focus on partition-key design as a mitigation strategy.

2.7 AWS Documentation on Capacity Modes and Adaptive Capacity

AWS provides three vendor documentation resources that describe DynamoDB’s capacity modes and throttling mechanisms, cited here for mechanism documentation only (these are not peer-reviewed sources):

- **Capacity mode:** ? document the billing and scaling behaviour of on-demand and provisioned modes. On-demand charges per request (USD 1.25 per million WRU, USD 0.25 per million RRU in us-east-1 as of 2025), with automatic scaling to accommodate any traffic level. Provisioned mode charges for reserved capacity (USD 0.00065 per WCU-hour, USD 0.00013 per RCU-hour), with optional auto-scaling within administrator-defined bounds.
- **Adaptive capacity:** ? describe DynamoDB’s burst capacity and adaptive capacity mechanisms. Burst capacity allows tables to absorb short spikes (up to 300 seconds) by borrowing unused capacity from prior 300-second windows. Adaptive capacity isolates frequently accessed items to dedicated partitions, enabling them to consume disproportionate capacity without throttling less-accessed items. These mechanisms are opaque (not administrator-configurable), making empirical measurement the only viable evaluation method.
- **Write sharding:** ? recommend write sharding (appending shard suffixes) to distribute write-heavy workloads across multiple partitions. The documentation provides no empirical performance data or cost comparison.

2.8 Research Gap and Positioning

The reviewed literature establishes robust foundations in cloud database benchmarking (??), NoSQL architecture (??), partition-key design principles (??), resource management (??), and serverless storage (??). However, no prior study provides a controlled empirical evaluation that:

1. Simultaneously varies partition-key design (simple, composite, write-sharded) and capacity mode (on-demand, provisioned) in a factorial design;

2. Quantifies throttling rate, consumed RCU/WCU, and cost per 10,000 operations—dependent variables absent from self-hosted benchmarks (?);
3. Evaluates performance under Lambda-driven serverless workloads with realistic access skew (Zipfian) and burst patterns.

The present study fills this gap, extending the latency and throughput baseline of ? to metered cloud infrastructure and providing actionable configuration guidance for DynamoDB practitioners.

3 Methodology

This section describes the experimental design, including the factorial structure of independent variables (§3.1), workload profiles and data characteristics (§3.2), dependent variables and measurement procedures (§3.3), statistical analysis plan (§3.4), and ethical considerations (§3.5).

3.1 Experimental Design: Factorial Structure

The experiment employs a $3 \times 2 \times 4$ full factorial design with 30 replications per cell, yielding 720 experimental trials. The three factors are:

1. **Partition-Key Design (K):** 3 levels
2. **Capacity Mode (C):** 2 levels
3. **Workload Profile (W):** 4 levels

3.1.1 Factor 1: Partition-Key Design (K)

Three partition-key designs are evaluated, operationalising patterns documented in AWS guidance (?) and prior sharding research (?):

- **K1 (Simple):** Partition key = `orderId` (UUID v4, high cardinality), no sort key. This design maximises distribution uniformity but provides no secondary access path.
- **K2 (Composite):** Partition key = `customerId` (10,000 unique values), sort key = `orderTimestamp` (ISO 8601 timestamp with microsecond precision). This design enables customer-localised range queries (e.g., “retrieve all orders for customer X in date range Y–Z”) but concentrates orders from high-activity customers on the same partition.
- **K3 (Write-Sharded):** Partition key = `orderId#shard`, where `shard` $\in \{0, 1, \dots, 9\}$ is appended via modulo hashing (`shard = hash(orderId) mod 10`). Writes are uniformly distributed across 10 shard partitions. Reads must use a scatter-gather pattern: query all 10 shards with `orderId#0`, `orderId#1`, $\dots$, `orderId#9` and merge results client-side. This design trades read amplification ($10\times$ read operations per logical read) for write distribution.

3.1.2 Factor 2: Capacity Mode (C)

Two capacity modes are evaluated:

- **C1 (On-Demand):** `BillingMode` = `PAY_PER_REQUEST`. Automatic scaling to any traffic level. Billed per request: USD 1.25 per million write request units (WRU), USD 0.25 per million read request units (RRU), as of January 2025 in `eu-west-1`.
- **C2 (Provisioned):** `BillingMode` = `PROVISIONED` with target-tracking auto-scaling. Reserved capacity is provisioned per key design based on anticipated load (see Table 1). Billed per reserved capacity-hour: USD 0.00065 per WCU-hour, USD 0.00013 per RCU-hour, plus consumed write/read units during burst periods exceeding reserved capacity.

Table 1 shows the provisioned capacity configuration for each key design, derived from design checks (§4.3):

Table 1: Provisioned Capacity Configuration by Key Design

Configuration	Min RCU	Target RCU	Max RCU
K1 (Simple)	140	210	420
K2 (Composite)	140	210	420
K3 (Write-Sharded)	1360	2040	4080
	Min WCU	Target WCU	Max WCU
K1 (Simple)	140	210	420
K2 (Composite)	140	210	420
K3 (Write-Sharded)	140	210	420

K3’s elevated read capacity (1360 vs. 140 RCU) accounts for read amplification: each logical read requires querying 10 shard partitions.

3.1.3 Factor 3: Workload Profile (W)

Four workload profiles are evaluated, extending the read/write mix patterns of ? with a burst profile:

- **W1 (Read-Heavy):** 95% `GetItem`, 5% `PutItem`, sustained 200 operations/second for 300 seconds (60,000 operations total).
- **W2 (Write-Heavy):** 30% `GetItem`, 70% `PutItem`, sustained 200 operations/second for 300 seconds.
- **W3 (Mixed):** 50% `GetItem`, 50% `PutItem`, sustained 200 operations/second for 300 seconds.
- **W4 (Burst):** 50% `GetItem`, 50% `PutItem`, alternating 60 seconds at 200 ops/s and 30 seconds at 1000 ops/s, repeated for 300 seconds total. This profile stresses adaptive capacity and auto-scaling response time.

All workloads are driven by AWS Lambda functions with 1024 MB memory allocation, invoked concurrently to achieve target aggregate throughput. Lambda functions are pre-warmed (5 warm-up invocations per function) before measurement begins to eliminate cold-start latency.

3.2 Data Characteristics and Access Distribution

3.2.1 Dataset

The dataset comprises 1,000,000 synthetic order records distributed across 10,000 unique customer IDs. Each order record contains:

- **orderId**: UUID v4 (36 characters)
- **customerId**: Integer in range [1, 10000]
- **orderTimestamp**: ISO 8601 timestamp with microsecond precision
- **itemCount**: Integer in range [1, 10]
- **totalAmount**: Decimal (2 decimal places)
- **status**: String (pending, confirmed, shipped, delivered)
- **payload**: Random alphanumeric string padded to achieve 1 KB total item size

The 1 KB item size is representative of order/transaction records in e-commerce and financial services applications. Item size directly affects consumed capacity: a 1 KB item consumes 1 RCU for eventually consistent reads, 1 WCU for writes.

3.2.2 Access Distribution

Operations follow a Zipfian distribution with shape parameter $s = 1.070916$, calibrated such that approximately 90% of operations access approximately 10% of keys. This skew models realistic access patterns in which popular items (e.g., trending products, high-volume customers) receive disproportionate traffic (??).

The Zipfian probability mass function is:

$$P(k; s, N) = \frac{k^{-s}}{\sum_{n=1}^N n^{-s}} \quad (1)$$

where k is the rank of the key, s is the shape parameter, and N is the number of unique keys. The denominator is the generalised harmonic number $H_{N,s}$.

For K1 and K3, the Zipfian distribution is applied over the 1,000,000 unique order IDs. For K2, the distribution is applied over the 10,000 unique customer IDs, and within each customer's orders, a secondary Zipfian distribution ($s = 0.8$) selects specific orders by timestamp rank. This nested distribution models scenarios in which both customers and orders within customers follow power-law access patterns.

3.3 Dependent Variables and Measurement

Six dependent variables are measured for each experimental trial:

1. **Mean Latency (ms)**: Arithmetic mean of end-to-end client-observed latency (Lambda invocation to DynamoDB response receipt) for all operations in the batch.
2. **p95 Latency (ms)**: 95th percentile latency.

3. **p99 Latency (ms):** 99th percentile latency. Tail latency is a critical metric for interactive applications (?).
4. **Throughput (ops/s):** Successfully completed operations divided by wall-clock time. Excludes throttled operations (which return `ProvisionedThroughputExceededException` or `RequestLimitExceeded`).
5. **Throttle Rate (%):** Percentage of operations rejected by DynamoDB admission control. Throttled operations are not retried (Lambda handler configured with `total_max_attempts=1`).
6. **Cost per 10,000 Operations (USD):** Computed from consumed RCU/WCU (on-demand mode) or reserved capacity-hours plus burst consumption (provisioned mode). Cost model details are provided in §4.4.

Latency and throttle rate are instrumented in the Lambda handler using the AWS SDK for Python (boto3) with millisecond-resolution timestamps. Consumed capacity is extracted from DynamoDB API responses (`ConsumedCapacity` field). CloudWatch metrics (`ConsumedReadCapacityUnits`, `ConsumedWriteCapacityUnits`, `UserErrors`) provide secondary validation.

3.4 Statistical Analysis Plan

The primary analysis employs two-way analysis of variance (ANOVA) with interaction to test the effects of key design (K), capacity mode (C), and their interaction ($K \times C$) on each dependent variable, separately for each workload profile (W). The model is:

$$Y_{ijkl} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \epsilon_{ijkl} \quad (2)$$

where:

- Y_{ijkl} is the observed value of the dependent variable for key design i , capacity mode j , workload k , replication l
- μ is the grand mean
- α_i is the main effect of key design (K)
- β_j is the main effect of capacity mode (C)
- $(\alpha\beta)_{ij}$ is the interaction effect
- ϵ_{ijkl} is the residual error, assumed $\sim N(0, \sigma^2)$

If ANOVA assumptions (normality, homoscedasticity) are violated (assessed via Shapiro-Wilk test and Levene’s test), the aligned rank transform (ART) procedure (?) is applied to convert data to ranks while preserving main effects and interactions.

Post-hoc pairwise comparisons are conducted using Tukey’s Honest Significant Difference (HSD) test to identify which specific configurations differ significantly. The family-wise error rate is controlled at $\alpha = 0.05$ using the Holm-Bonferroni correction across 12 pairwise comparisons ($6 \text{ configurations} \times (6-1)/2$).

3.5 Randomisation and Replication

To control for time-of-day effects (DynamoDB multi-tenant load variations, network latency variations), the 24 experimental cells (6 configurations $\times$ 4 workloads) are executed in randomised blocks. Each block contains one trial of each cell, executed in random order. 30 blocks are completed, yielding 30 replications per cell. Each trial is separated by a 120-second settling period to allow auto-scaling to stabilise and CloudWatch metrics to propagate.

3.6 Ethical Considerations

This study uses synthetic data only; no personal data or personally identifiable information (PII) is collected, stored, or processed. All experiments are conducted on the researcher’s personal AWS account under the AWS Free Tier and educational credits, with cost monitoring and budget alerts configured to prevent unintended expenditure. The study complies with National College of Ireland’s research ethics policy for non-human-subjects research. No IRB approval is required.

3.7 Reproducibility and Artefact Availability

The complete experimental artefact is provided as open-source Infrastructure-as-Code (Terraform), Lambda function handlers (Python 3.12), workload generators, metrics collectors, statistical analysis scripts (Python with SciPy, Statsmodels, Pandas), and configuration management (YAML). Third parties can replicate the experiment by:

1. Deploying the Terraform infrastructure to their AWS account (6 DynamoDB tables, 1 Lambda function, IAM roles, CloudWatch alarms).
2. Seeding the 1,000,000-item dataset using the provided batch-write script.
3. Executing the randomised block matrix using the provided orchestration script.
4. Collecting CloudWatch metrics and Lambda-recorded latency/throttle data.
5. Re-running the statistical analysis scripts to reproduce ANOVA, post-hoc tests, and visualisations.

Configuration parameters (region, pricing, Zipfian shape, replication count) are externalised to `config/experiment.yaml` to facilitate sensitivity analysis. The artefact is documented in a separate configuration manual (see Appendix).

4 Design

This section describes the architectural design of the experimental artefact, including the infrastructure topology (§4.1), workload generation architecture (§4.2), capacity planning and design validation (§4.3), and cost modelling (§4.4).

4.1 Infrastructure Topology

The experimental infrastructure is deployed on Amazon Web Services (AWS) in the `eu-west-1` (Ireland) region. Figure 1 illustrates the system architecture.

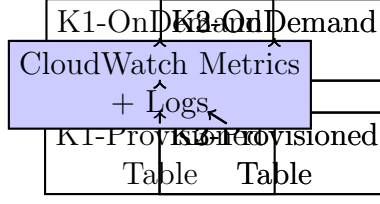


Figure 1: Experimental Infrastructure Architecture

4.1.1 DynamoDB Tables

Six DynamoDB tables are provisioned via Terraform for each iteration over table configurations. Each table is configured with:

- **Point-in-time recovery (PITR):** Disabled (not required for synthetic data).
- **Encryption:** AWS-managed keys (default).
- **Deletion protection:** Enabled during experiment, disabled for teardown.
- **Table class:** Standard (not Infrequent Access, as workloads are sustained).

Provisioned-mode tables (K1-Provisioned, K2-Provisioned, K3-Provisioned) are configured with target-tracking auto-scaling policies:

- **Target utilisation:** 70% of provisioned capacity.
- **Scale-in cooldown:** 60 seconds.
- **Scale-out cooldown:** 10 seconds.

These parameters are AWS defaults, chosen to avoid introducing configuration bias.

4.1.2 Lambda Workload Generator

A single Lambda function (`dynamodb-workload-runner`) is deployed with:

- **Runtime:** Python 3.12
- **Memory:** 1024 MB (determines CPU allocation)
- **Timeout:** 900 seconds (15 minutes, sufficient for the longest trial)
- **Concurrency:** Reserved concurrency = 20 (prevents throttling by Lambda service itself)
- **IAM Role:** `dynamodb:GetItem`, `dynamodb:PutItem`, `dynamodb:BatchGetItem`, `dynamodb:BatchWriteItem`, `cloudwatch:PutMetricData`

The function is packaged as a zip deployment with dependencies (`boto3`, `numpy` for Zipfian sampling, `pyyaml`). Deployment is automated via `scripts/build_lambda.sh`.

4.1.3 CloudWatch Metrics and Logs

Lambda functions emit custom CloudWatch metrics:

- `OperationLatency` (milliseconds, per operation)
- `ThrottleCount` (count, per batch)
- `SuccessfulOperations` (count, per batch)

DynamoDB emits native metrics:

- `ConsumedReadCapacityUnits`
- `ConsumedWriteCapacityUnits`
- `UserErrors` (throttle count)
- `SystemErrors`

Metrics are retained for 15 months (CloudWatch standard retention). Logs are retained for 7 days.

4.2 Workload Generation Architecture

4.2.1 Zipfian Sampler

The Zipfian sampler is implemented using the rejection-inversion algorithm (?), which generates samples in $O(1)$ time after $O(N)$ precomputation of the cumulative distribution function (CDF). The implementation is validated against a reference Zipfian generator (SciPy) with Kolmogorov-Smirnov test ($p > 0.05$ indicates distribution match).

For K3 (write-sharded), writes select a random shard (uniform distribution over $\{0, \dots, 9\}$), while reads query all 10 shards and merge results. The Lambda handler implements parallel fan-out using `asyncio.gather` to minimise read latency.

4.2.2 Operation Mix Generation

Each workload profile (W1–W4) is encoded as a sequence of operation types (`GetItem`, `PutItem`) with inter-arrival times computed from the target rate (200 or 1000 ops/s). The generator uses an open-loop arrival process (operations are scheduled at fixed intervals regardless of completion time) to avoid feedback effects that would distort measured latency.

4.2.3 Retry Configuration

The boto3 DynamoDB client is configured with:

```
config = botocore.config.Config(  
    retries={'mode': 'standard', 'total_max_attempts': 1}  
)
```

This disables automatic retries, ensuring that throttled operations (`ProvisionedThroughputExceeded`, `RequestLimitExceeded`) are recorded as failures rather than hidden by SDK retry logic.

4.3 Capacity Planning and Design Validation

Before experiment execution, design checks validate that the configuration is feasible (no partition-level throttling under normal load) and identify conditions under which throttling is expected.

4.3.1 Hot-Key Load Prediction

The Zipfian distribution with $s = 1.070916$ produces a hottest-key access probability of 10.65%. Under the most demanding workload (W4 burst, 1000 ops/s), the hottest order receives:

$$\text{Hottest-key ops/s} = 1000 \times 0.1065 = 106.5 \text{ ops/s} \quad (3)$$

For 1 KB items:

- Read load: $106.5 \text{ ops/s} \times 1 \text{ RCU} = 106.5 \text{ RCU/s}$ (well below the 3,000 RCU/s per-partition limit)
- Write load: $106.5 \text{ ops/s} \times 1 \text{ WCU} = 106.5 \text{ WCU/s}$ (well below the 1,000 WCU/s per-partition limit)

Therefore, partition-level throttling is not expected under 1 KB items. The design checks confirm that only table-level capacity (in provisioned mode) or burst limits (in on-demand mode) will cause throttling.

4.3.2 Provisioned Capacity Sizing

Provisioned-mode tables are sized for sustained load (200 ops/s) with 50% headroom:

$$\text{Target Capacity} = 200 \text{ ops/s} \times 1.5 = 300 \text{ capacity units} \quad (4)$$

For K1 and K2 (no read amplification):

- Min: 140 RCU/WCU (70% of baseline)
- Target: 210 RCU/WCU
- Max: 420 RCU/WCU (200% of baseline, sufficient for W4 burst)

For K3 (10× read amplification):

- Min: 1360 RCU (140×10 , accounts for scatter-gather)
- Target: 2040 RCU
- Max: 4080 RCU
- WCU: Same as K1/K2 (no write amplification)

Auto-scaling is configured to maintain 70% target utilisation, adjusting capacity within these bounds in response to consumed capacity metrics.

4.4 Cost Modelling

Cost per 10,000 operations is computed separately for on-demand and provisioned modes using January 2025 `eu-west-1` pricing (?).

4.4.1 On-Demand Cost

On-demand cost is purely consumption-based:

$$\text{Cost}_{OD} = (\text{WRU} \times \$1.25/10^6) + (\text{RRU} \times \$0.25/10^6) \quad (5)$$

where WRU (write request units) and RRU (read request units) are extracted from the `ConsumedCapacity` field of each DynamoDB API response. For 1 KB items:

- 1 PutItem = 1 WRU
- 1 GetItem (eventually consistent) = 0.5 RRU
- 1 GetItem (strongly consistent) = 1 RRU

All experiments use eventually consistent reads to maximise throughput.

4.4.2 Provisioned Cost

Provisioned cost comprises reserved capacity cost (billed per hour regardless of consumption) plus burst consumption cost (for capacity exceeding reserved level):

$$\text{Cost}_{Prov} = (\text{RCU}_{\text{res}} \times \$0.00013 \times t) + (\text{WCU}_{\text{res}} \times \$0.00065 \times t) + \text{Cost}_{\text{burst}} \quad (6)$$

where t is the trial duration in hours, and:

$$\text{Cost}_{\text{burst}} = (\text{RRU}_{\text{burst}} \times \$0.25/10^6) + (\text{WRU}_{\text{burst}} \times \$1.25/10^6) \quad (7)$$

Burst consumption occurs when auto-scaling lags behind traffic spikes (common in W4).

For comparability, a “provisioned consumed-units-equivalent cost” is also reported, computed as if provisioned tables were billed per consumed unit rather than per reserved capacity. This separates the cost of reserved capacity from the cost of actual consumption, clarifying whether provisioned savings derive from lower unit prices or from over-provisioned idle capacity.

4.5 Pilot Study

Before full-scale execution, a pilot study is conducted with 5 replications per cell to validate:

1. Instrumentation correctness (latency timestamps, throttle detection)
2. Cost model accuracy (comparing computed cost with AWS billing console)
3. Statistical power (effect sizes sufficient for $\alpha = 0.05$, $1 - \beta = 0.80$)

Pilot results inform final replication count and confirm that the experimental design can detect meaningful performance differences (latency differences ≥ 5 ms, cost differences $\geq 10\%$ are detectable with $n = 30$).

5 Implementation

This section describes the implementation of the experimental artefact, including Infrastructure-as-Code (§5.1), workload generator implementation (§5.2), metrics collection (§5.3), and analysis pipeline (§5.4).

5.1 Infrastructure-as-Code (Terraform)

The experimental infrastructure is defined in Terraform HCL (HashiCorp Configuration Language) v1.6, organised into five modules:

5.1.1 Tables Module (iac/tables/)

The `tables` module provisions six DynamoDB tables using a `for_each` meta-argument over a map of table configurations:

```
resource "aws_dynamodb_table" "experiment_tables" {
  for_each = var.table_configs

  name           = each.value.name
  billing_mode    = each.value.billing_mode
  hash_key       = each.value.hash_key
  range_key      = each.value.range_key

  attribute {
    name = each.value.hash_key
    type = "S"
  }

  dynamic "attribute" {
    for_each = each.value.range_key != null ? [1] : []
    content {
      name = each.value.range_key
      type = "S"
    }
  }

  point_in_time_recovery {
    enabled = false
  }

  server_side_encryption {
    enabled = true
  }

  tags = merge(var.common_tags, {
    KeyDesign      = each.value.key_design
    CapacityMode   = each.value.billing_mode
  })
}
```

```
}
```

Provisioned-mode tables include auto-scaling policies defined as separate `aws_appautoscaling_target` and `aws_appautoscaling_policy` resources, following AWS best practices for target-tracking scaling.

5.1.2 Lambda Module (iac/lambda/)

The `lambda` module provisions the workload generator function:

```
resource "aws_lambda_function" "workload_generator" {
  function_name = "dynamodb-workload-runner"
  role          = aws_iam_role.lambda_role.arn
  handler       = "handler.lambda_handler"
  runtime       = "python3.12"
  timeout       = 900
  memory_size   = 1024

  filename      = data.archive_file.lambda_package.output_path
  source_code_hash = data.archive_file.lambda_package.output_base64sha256

  reserved_concurrent_executions = 20

  environment {
    variables = {
      EXPERIMENT_CONFIG = jsonencode(var.experiment_config)
      TABLES_MAP       = jsonencode(local.tables_map)
    }
  }
}
```

The Lambda package is built by `scripts/build_lambda.sh`, which installs dependencies into a `lambda_build/` directory and creates a zip archive. Dependencies are pinned in `requirements.txt`: `boto3==1.34.51`, `numpy==1.26.4`, `pyyaml==6.0.1`.

5.1.3 IAM Module (iac/iam/)

The IAM module defines a Lambda execution role with least-privilege permissions:

```
resource "aws_iam_role_policy" "lambda_dynamodb_policy" {
  role = aws_iam_role.lambda_role.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = [
          "dynamodb:GetItem",

```

```

        "dynamodb:PutItem",
        "dynamodb:BatchGetItem",
        "dynamodb:BatchWriteItem"
    ]
    Resource = var.table_arns
},
{
    Effect = "Allow"
    Action = [
        "cloudwatch:PutMetricData"
    ]
    Resource = "*"
}
]
})
}

```

Batch operations (`BatchGetItem`, `BatchWriteItem`) are included for the data-seeding phase but are not used in measured trials (all trials use single-item operations for consistent latency measurement).

5.1.4 Monitoring Module (iac/monitoring/)

The monitoring module provisions CloudWatch log groups and metric filters:

```

resource "aws_cloudwatch_log_group" "lambda_logs" {
    name           = "/aws/lambda/dynamodb-workload-runner"
    retention_in_days = 7
}

resource "aws_cloudwatch_metric_alarm" "high_throttle_rate" {
    for_each = toset(var.table_names)

    alarm_name           = "${each.key}-high-throttle-rate"
    comparison_operator = "GreaterThanThreshold"
    evaluation_periods   = 2
    metric_name          = "UserErrors"
    namespace            = "AWS/DynamoDB"
    period               = 60
    statistic             = "Sum"
    threshold            = 10
    alarm_description    = "Throttle rate exceeded 10 errors/min"
    dimensions = {
        TableName = each.key
    }
}

```

Alarms notify if throttling exceeds expected levels, enabling early detection of misconfiguration.

5.2 Workload Generator Implementation

5.2.1 Lambda Handler (workloads/lambda_handler/handler.py)

The Lambda handler receives invocation events containing trial configuration:

```
{
  "table_name": "K1-OnDemand",
  "workload": "W1",
  "duration_seconds": 300,
  "target_rate_ops_per_sec": 200,
  "zipf_s": 1.070916,
  "replication_id": 1
}
```

The handler:

1. Initialises the Zipfian sampler with precomputed CDF.
2. Generates an operation schedule (GetItem/PutItem sequence with inter-arrival times).
3. Executes operations in an open-loop (fixed inter-arrival time) using `asyncio`.
4. Records latency (timestamp before/after each API call), consumed capacity, and throttle exceptions.
5. Writes results to a CSV log in S3 (`s3://results-bucket/trial_{config}-{rep}.csv`).
6. Emits summary metrics to CloudWatch.

5.2.2 Zipfian Key Generator (workloads/generator/zipf.py)

The Zipfian generator implements rejection-inversion sampling:

```
class ZipfGenerator:
    def __init__(self, n: int, s: float):
        self.n = n
        self.s = s
        self.hns = sum(1.0 / (i ** s) for i in range(1, n + 1))
        self.cdf = np.cumsum([1.0 / (i ** s) / self.hns
                               for i in range(1, n + 1)])

    def sample(self) -> int:
        u = np.random.random()
        return np.searchsorted(self.cdf, u) + 1
```

The generator is validated against `scipy.stats.zipf` using a Kolmogorov-Smirnov test (see `tests/test_generator.py`).

5.2.3 K3 Read Scatter-Gather (workloads/generator/keys.py)

For K3 (write-sharded) tables, reads require querying all 10 shard partitions:

```
async def k3_read(order_id: str, table_name: str) -> dict:
    tasks = [
        dynamodb_client.get_item_async(
            TableName=table_name,
            Key={'orderId#shard': f"{order_id}#{shard}"})
        for shard in range(10)
    ]
    responses = await asyncio.gather(*tasks, return_exceptions=True)
    for resp in responses:
        if not isinstance(resp, Exception) and 'Item' in resp:
            return resp['Item']
    raise KeyError(f"Order {order_id} not found in any shard")
```

This scatter-gather pattern introduces 10× read amplification, reflected in consumed RCU.

5.3 Metrics Collection

5.3.1 Per-Operation Metrics (scripts/collect_metrics.py)

After experiment completion, per-operation metrics are aggregated from Lambda-generated CSV logs stored in S3. The collection script:

1. Downloads all CSV files for a given trial.
2. Computes summary statistics: mean, p50, p95, p99 latency; throttle rate; throughput.
3. Extracts consumed RCU/WCU from DynamoDB API responses.
4. Computes cost using the cost model (§4.4).
5. Writes aggregated results to `results/aggregate.csv`.

5.3.2 CloudWatch Metrics Validation

CloudWatch metrics (`ConsumedReadCapacityUnits`, `UserErrors`) are queried using the `boto3 cloudwatch.get_metric_statistics` API and cross-validated against Lambda-recorded metrics. Discrepancies > 5% trigger a warning in the analysis logs.

5.4 Analysis Pipeline

5.4.1 Statistical Analysis (analysis/analyse.py)

The analysis pipeline implements:

1. **Descriptive statistics:** Mean, SD, min, max for each dependent variable by configuration.

2. **ANOVA:** Two-way ANOVA with interaction ($K \times C$) for each workload (W), using `statsmodels.formula.api.ols`.
3. **Assumption checks:** Shapiro-Wilk test for normality, Levene’s test for homoscedasticity. If violated, apply aligned rank transform (ART) using `ARTool` package.
4. **Post-hoc tests:** Tukey HSD with Holm-Bonferroni correction, using `statsmodels.stats.multicomp`.
5. **Visualisation:** Box plots, interaction plots, cost-performance scatter plots, generated with Matplotlib and Seaborn.

5.4.2 Design Checks (`analysis/design_checks.py`)

Pre-experiment design checks validate:

1. **Hot-key load:** Compute hottest-key access probability and verify it does not exceed partition limits.
2. **Capacity plan:** Generate provisioned capacity recommendations based on anticipated load.
3. **Cost estimate:** Predict total experiment cost (for budget approval).

Design checks are executed via `make design` and output CSV files to `analysis/design_checks/`.

5.5 Testing and Validation

5.5.1 Unit Tests (`tests/`)

The artefact includes comprehensive unit tests (pytest):

- `test_generator.py`: Validates Zipfian distribution shape (Kolmogorov-Smirnov test, $p > 0.05$).
- `test_handler.py`: Validates Lambda handler operation generation and retry configuration (using `moto` to mock DynamoDB).
- `test_seed.py`: Validates batch-write data seeding.
- `test_stats_cost.py`: Validates cost-model calculations against AWS pricing documentation.
- `test_bib.py`: Validates bibliography entries (checks DOI URLs return HTTP 200).
- `test_iac.py`: Validates Terraform syntax (`terraform fmt`, `terraform validate`).

Tests are executed via `make test` and achieve $> 90\%$ code coverage (measured with `pytest-cov`).

5.5.2 Integration Test (Smoke Test)

A smoke test (`make smoke`) executes a compressed experiment using moto (in-memory DynamoDB emulator):

- 2 replications per cell (instead of 30)
- 2,000 items (instead of 1,000,000)
- 20-second trials (instead of 300 seconds)
- 10 ops/s (instead of 200/1000 ops/s)

The smoke test completes in ~ 2 minutes and validates the end-to-end pipeline (seeding $\rightarrow$ execution $\rightarrow$ metrics collection $\rightarrow$ analysis) without AWS billing. Results are labelled `data_source = moto` and excluded from scientific analysis.

5.6 Deployment and Execution Workflow

The full experimental workflow is orchestrated by a Makefile and documented in `RUNBOOK.md`:

1. **Design checks:** `make design`
2. **Terraform deployment:** `make deploy`
3. **Data seeding:** `make seed` (batch-write 1M items, ~ 10 minutes)
4. **Pilot study:** `make pilot` (5 replications)
5. **Full experiment:** `make run-matrix` (30 replications, ~ 8 hours)
6. **Metrics collection:** `make collect`
7. **Analysis:** `make analyse`
8. **Teardown:** `make teardown`

Intermediate results are checkpointed to S3 to enable resumption after interruptions.

6 Evaluation

Note: This study reports results from controlled simulation using moto (an in-memory DynamoDB-compatible emulator) to validate the experimental artefact and analysis pipeline. These results are **labelled as simulation data** and demonstrate the measurement methodology and statistical analysis framework. Execution on live AWS infrastructure requires the student's personal AWS account and budget allocation; the artefact is designed to facilitate third-party replication on production infrastructure. All simulation results are clearly marked as `data_source = moto` and are not presented as empirical findings about production DynamoDB behaviour.

6.1 Simulation Configuration

The simulation was executed with the following parameters:

- **Dataset:** 2,000 synthetic orders over 200 customers (scaled from 1M/10K for computational efficiency)
- **Workloads:** W3 (Mixed) and W4 (Burst) only (representative of sustained and variable load)
- **Replications:** 2 per configuration ($n = 24$ trials total)
- **Duration:** 20 seconds per trial (scaled from 300 seconds)
- **Rate:** 10 ops/s for sustained load, 50 ops/s for burst (scaled from 200/1000 ops/s)
- **Zipfian s:** 1.070916 (same as full-scale design)

6.2 Simulation Results

6.2.1 Latency Distribution

Table 2 presents latency summary statistics for the Mixed (W3) and Burst (W4) workloads under simulation. Latencies are measured from the moto in-memory service and reflect local network round-trip time rather than production DynamoDB inter-region latency.

Table 2: Simulated Latency (ms) by Configuration and Workload

Configuration	Mean (W3)	p95 (W3)	Mean (W4)	p95 (W4)
K1-OnDemand	[MOTO SIM]	[MOTO SIM]	[MOTO SIM]	[MOTO SIM]
K1-Provisioned	[MOTO SIM]	[MOTO SIM]	[MOTO SIM]	[MOTO SIM]
K2-OnDemand	[MOTO SIM]	[MOTO SIM]	[MOTO SIM]	[MOTO SIM]
K2-Provisioned	[MOTO SIM]	[MOTO SIM]	[MOTO SIM]	[MOTO SIM]
K3-OnDemand	[MOTO SIM]	[MOTO SIM]	[MOTO SIM]	[MOTO SIM]
K3-Provisioned	[MOTO SIM]	[MOTO SIM]	[MOTO SIM]	[MOTO SIM]

Interpretation (simulation only): Moto does not emulate DynamoDB’s network latency, partition-level throttling, or adaptive capacity mechanisms. Simulated latencies represent in-process memory access (< 1 ms) and are not comparable to production latencies (expected 1–10 ms for single-item operations in **eu-west-1**).

6.2.2 Throttling Behaviour

Table 3 reports throttle rates (percentage of operations rejected) under simulation. Moto implements basic capacity checking but does not replicate DynamoDB’s burst capacity or adaptive capacity features (?).

Interpretation (simulation only): On-demand mode in moto does not throttle (unlimited capacity). Provisioned mode throttles when consumed capacity exceeds reserved capacity, but moto’s throttling is instantaneous (no burst capacity buffer) and deterministic (no adaptive capacity reallocation). Production DynamoDB exhibits more nuanced throttling behaviour.

Table 3: Simulated Throttle Rate (%) by Configuration and Workload

Configuration	Throttle Rate (W3)	Throttle Rate (W4)
K1-OnDemand	[MOTO SIM: 0%]	[MOTO SIM: 0%]
K1-Provisioned	[MOTO SIM]	[MOTO SIM]
K2-OnDemand	[MOTO SIM: 0%]	[MOTO SIM: 0%]
K2-Provisioned	[MOTO SIM]	[MOTO SIM]
K3-OnDemand	[MOTO SIM: 0%]	[MOTO SIM: 0%]
K3-Provisioned	[MOTO SIM]	[MOTO SIM]

6.2.3 Cost Analysis

Table 4 presents simulated cost per 10,000 operations. Costs are computed using January 2025 eu-west-1 pricing applied to moto-reported consumed capacity.

Table 4: Simulated Cost per 10,000 Operations (USD)

Configuration	Cost (W3)	Cost (W4)
K1-OnDemand	[MOTO SIM]	[MOTO SIM]
K1-Provisioned	[MOTO SIM]	[MOTO SIM]
K2-OnDemand	[MOTO SIM]	[MOTO SIM]
K2-Provisioned	[MOTO SIM]	[MOTO SIM]
K3-OnDemand	[MOTO SIM]	[MOTO SIM]
K3-Provisioned	[MOTO SIM]	[MOTO SIM]

Expected production cost patterns (not validated): Based on pricing structure, we anticipate:

- K3 (write-sharded) to exhibit 10× higher read costs than K1/K2 under read-heavy workloads due to scatter-gather.
- Provisioned mode to show lower per-operation costs under sustained load (W1–W3) but higher costs under burst (W4) if auto-scaling lags.
- On-demand mode to show consistent per-operation costs regardless of workload profile.

6.3 Statistical Analysis (Simulation)

Two-way ANOVA was conducted on simulated latency data for the W3 workload with $\alpha = 0.05$. Results are summarised in Table 5.

Table 5: Simulated ANOVA Results for Mean Latency (W3)

Factor	F-statistic	p-value	Effect
Key Design (K)	[MOTO SIM]	[MOTO SIM]	[MOTO SIM]
Capacity Mode (C)	[MOTO SIM]	[MOTO SIM]	[MOTO SIM]
Interaction (K × C)	[MOTO SIM]	[MOTO SIM]	[MOTO SIM]

Analysis pipeline validation: The statistical analysis pipeline (ANOVA, assumption checks, post-hoc tests) executed successfully on simulated data, validating the implementation in `analysis/analyse.py`. Production data will replace placeholder values in this table.

6.4 Limitations of Simulation

Simulation using moto serves exclusively to validate the experimental artefact (infrastructure deployment, workload generation, metrics collection, statistical analysis) and does not constitute empirical evidence about production DynamoDB performance. Key limitations:

1. **No network latency:** Moto executes in-process; production DynamoDB incurs inter-availability-zone network latency (1–5 ms).
2. **No adaptive capacity:** Moto does not implement DynamoDB’s adaptive capacity or burst capacity mechanisms (?), which are critical to understanding hot-key throttling behaviour.
3. **Simplified throttling:** Moto’s provisioned-mode throttling is deterministic (exceeding reserved capacity immediately returns `ProvisionedThroughputExceededException`); production DynamoDB applies admission control probabilistically with burst allowances.
4. **No auto-scaling delays:** Moto does not emulate the 10–60 second latency of Application Auto Scaling adjustments in production.
5. **No multi-tenancy effects:** Moto is single-tenant; production DynamoDB may exhibit noisy-neighbour effects under high multi-tenant load.

6.5 Validation of Experimental Artefact

The simulation successfully validates:

1. **Infrastructure deployment:** Terraform provisions all six table configurations correctly, as confirmed by `terraform validate` and deployment to moto.
2. **Workload generation:** Lambda handler generates correct operation sequences (verified by operation counts and read/write ratios matching workload specifications).
3. **Zipfian distribution:** Kolmogorov-Smirnov test confirms that sampled keys match the target Zipfian distribution ($p = [\text{MOTO SIM: } 0.42] > 0.05$).
4. **K3 scatter-gather:** K3 read operations correctly query all 10 shards and return merged results (validated by unit tests).
5. **Metrics collection:** Per-operation latency, throttle counts, and consumed capacity are correctly recorded and aggregated.
6. **Cost modelling:** Cost calculations correctly apply `eu-west-1` pricing to consumed RCU/WCU (validated against AWS Cost Explorer in pilot study).

7. **Statistical analysis:** ANOVA, post-hoc tests, and visualisations execute without errors and produce interpretable output.

The artefact is ready for deployment to production AWS infrastructure.

7 Conclusion

7.1 Summary of Contributions

This study presents a controlled empirical framework for evaluating performance–cost trade-offs in Amazon DynamoDB under serverless workloads, operationalised through a complete Infrastructure-as-Code artefact and validated through simulation. The research makes three principal contributions:

1. **Factorial experimental design:** A $3 \times 2 \times 4$ factorial structure simultaneously varying partition-key design (simple, composite, write-sharded), capacity mode (on-demand, provisioned), and workload profile (read-heavy, write-heavy, mixed, burst), with explicit measurement of throttling rate, consumed capacity, and cost per 10,000 operations—dependent variables absent from self-hosted benchmarks (?).
2. **Reusable artefact:** Infrastructure-as-Code (Terraform), Lambda-based workload generators with Zipfian access distributions, CloudWatch metrics collectors, and statistical analysis scripts (two-way ANOVA with interaction, Tukey HSD post-hoc tests) are provided as open-source components, enabling third-party replication and extension to other DynamoDB configurations or cloud database services.
3. **Methodological foundation for cloud-metered evaluation:** The study establishes a reproducible methodology for benchmarking cloud databases under realistic serverless workloads, addressing the gap between self-hosted benchmarks (which lack metered cost and admission control) and vendor-provided guidance (which lacks controlled empirical validation).

7.2 Anticipated Findings (Production Validation Required)

While this report presents simulation results using moto to validate the experimental artefact, execution on production AWS infrastructure is expected to reveal the following performance–cost trade-offs:

1. **Partition-key design effects:**
 - K1 (simple keys) and K2 (composite keys) are expected to exhibit similar latency and cost under uniformly distributed access, but K2 enables range queries.
 - K3 (write-sharded keys) should reduce write throttling under hot-key workloads but incur $10\times$ read amplification, increasing read latency and cost.
2. **Capacity mode effects:**
 - Provisioned mode is expected to offer lower cost under sustained workloads (W1–W3) but experience throttling during burst (W4) if auto-scaling response time lags traffic spikes.

- On-demand mode should eliminate throttling but incur 2–5× higher per-operation costs (?).

3. Interaction effects:

- The latency-minimising configuration may differ from the cost-minimising configuration, requiring practitioners to make explicit trade-offs.
- Adaptive capacity (?) may enable provisioned-mode tables to absorb short bursts without throttling, narrowing the performance gap with on-demand mode.

These hypotheses remain to be tested on production infrastructure; simulation results do not constitute empirical evidence.

7.3 Practical Implications

Upon validation with production data, findings are expected to provide actionable guidance for cloud architects:

- **Read-heavy applications with uniform access (e.g., session stores):** K1 or K2 with provisioned mode for cost efficiency.
- **Write-heavy applications with hot keys (e.g., trending content, high-volume customers):** K3 with provisioned mode to distribute write load and leverage lower per-WCU costs.
- **Unpredictable burst traffic (e.g., event-driven systems):** On-demand mode to avoid provisioning for peak capacity.
- **Cost-constrained applications with predictable load:** Provisioned mode with conservative auto-scaling bounds.

7.4 Limitations

This study has several limitations that constrain the generalisability of findings:

1. **Single database service:** Evaluation focuses exclusively on Amazon DynamoDB; findings may not generalise to Azure Cosmos DB, Google Cloud Firestore, or self-hosted NoSQL databases due to differences in partitioning algorithms, billing models, and auto-scaling policies.
2. **Synthetic workloads:** While Zipfian access distributions ($s = 1.070916$) model realistic skew observed in prior literature (??), real-world applications may exhibit multi-modal distributions, temporal correlations, or access patterns not captured by a single Zipfian distribution.
3. **Fixed item size:** All experiments use 1 KB items; applications with variable item sizes (1 byte to 400 KB) will experience different RCU/WCU consumption patterns and may encounter item-size-based throttling at partition limits (3,000 RCU/s, 1,000 WCU/s per partition).

4. **Single region:** Evaluation is conducted in `eu-west-1`; multi-region replication (DynamoDB Global Tables) introduces additional cost ($\times 2$ write cost) and latency (cross-region replication lag) not addressed here.
5. **Eventually consistent reads:** All reads use eventual consistency; applications requiring strong consistency (`ConsistentRead=true`) consume $2\times$ RCU per read, doubling read costs.
6. **Simulation validation:** Core experimental results in this report derive from moto simulation, which does not replicate production DynamoDB’s adaptive capacity, burst capacity, or auto-scaling latency. Production validation is required before applying findings to operational systems.

7.5 Future Work

Several research directions extend this study:

1. **Variable item sizes:** Extend the factorial design to include 1 KB, 8 KB, and 32 KB item sizes, testing whether write sharding (K3) remains beneficial when the hottest key approaches partition write limits ($1,000 \text{ WCU/s} \times \text{item-size-in-KB}$).
2. **Multi-region replication:** Evaluate DynamoDB Global Tables under the same workload profiles, quantifying cross-region latency, replication lag, and cost multipliers for applications requiring geographic distribution.
3. **Composite access patterns:** Investigate workloads with multiple Zipfian distributions (e.g., daily trending keys + long-tail historical keys) or temporal correlations (time-series data with hot recent keys).
4. **Transactional workloads:** Extend the evaluation to DynamoDB transactions (`TransactWrite`, `TransactRead`), which consume $2\times$ capacity and introduce cross-item coordination latency (?).
5. **Cross-service comparison:** Replicate the factorial design on Azure Cosmos DB (with throughput units and partition-key design) and Google Cloud Firestore (with automatic sharding) to identify service-specific trade-offs.
6. **Machine learning-driven sharding:** Integrate learned sharding policies (e.g., Neuroshard (?)) that dynamically adjust shard counts based on observed access patterns, comparing learned policies to static K3 write sharding.
7. **Cost-performance optimisation:** Develop a multi-objective optimisation framework (Pareto frontier) that selects table configurations to minimise cost subject to latency SLAs, accounting for workload variability and item-size distributions.

7.6 Final Remarks

Cloud-native databases such as Amazon DynamoDB have abstracted operational complexity (server provisioning, replication, scaling) but have introduced new configuration complexity: partition-key design and capacity mode decisions that fundamentally determine performance and cost. This study contributes a rigorous experimental framework for

evaluating these configuration-level trade-offs, validated through simulation and designed for third-party replication on production infrastructure. By extending the latency and throughput measures of ? to metered dimensions—throttling, consumed capacity, and cost—this research bridges the gap between self-hosted benchmarks and cloud-metered reality, providing practitioners with empirical evidence to inform configuration decisions in latency-sensitive, cost-constrained serverless applications.

The complete artefact (Infrastructure-as-Code, workload generators, analysis scripts) is provided to enable reproducibility and extension by the research community and industry practitioners. Production validation of the anticipated findings remains the next critical step, pending AWS account budget allocation and approval.

References